Tarn

Architecture Guide

Permanent, encrypted, user-owned data infrastructure

April 2026

Contents

Executive summary

Tarn is a platform for permanent, encrypted, user-owned data. It is infrastructure — not an end-user product — that any application can build on to give its users data that is theirs in a way that survives the app, the platform, and the company.

Three properties define Tarn:

- **Permanent.** User data is stored on Arweave, a permanent decentralized ledger. Once written, it is there forever, signed and timestamped, independent of any company's continued operation.
- **Encrypted.** All content is encrypted client-side before it leaves the user's device. The Tarn service never sees plaintext, never holds the keys, and cannot be coerced into producing user content even under legal compulsion.
- **User-owned.** Authentication and decryption are derived from secrets the user holds: their email + password, plus a recovery phrase issued at signup. There is no provider in the trust loop. If Tarn disappears, the user can still decrypt their data from raw Arweave.

The user's content layer is augmented by a sharing layer that lets users selectively share specific items with specific people, end-to-end encrypted, without exposing their identity graph to the storage provider.

This document is a human-readable architecture guide. It is intended for technical readers who want to understand what Tarn is, how it is shaped, and what trade-offs are baked into the design. For the formal protocol specification, see *TARN_PROTOCOL.md* in the repository.

1. Why Tarn exists

Most consumer software treats user data as a side effect of providing a service. The data lives in the company's database, encrypted (if at all) with keys the company controls. The implicit contract is: *trust the company.* Trust them not to read your messages. Trust them to stay in business. Trust them not to pivot, sell to a worse owner, or be coerced by a government. Trust them, year after year, indefinitely.

When that trust breaks — and it eventually does, for many users — the data is held hostage. Photos in Facebook. Email in Google. Notes in Evernote. Medical history in a portal whose terms you never re-read. Even when companies act in good faith, services shut down (Google Reader, Vine, every product Yahoo touched), and the data goes with them.

Tarn flips the model. The data layer is not the company's database; it is the user's. Encryption happens client-side with keys the user holds. Permanence is provided by Arweave, not by Tarn's continued operation. If Tarn the service stops existing tomorrow, every user can still recover their data with nothing more than their credentials.

The result: applications built on Tarn inherit a stronger privacy and ownership story than they could plausibly claim alone. The application developer doesn't have to be trustworthy — the cryptography is.

2. Trust model and design principles

Tarn's design is shaped by six principles. Every architectural decision ultimately answers to these.

Zero knowledge

The Tarn API never sees plaintext. All encryption happens on the client before bytes leave the user's device. Even with full administrative access to the Tarn database, an operator cannot decrypt user content.

No infrastructure dependency for recovery

If Tarn disappears, users can still decrypt their data. Tarn's database is a cache that can be rebuilt from raw Arweave, and any client with the user's credentials can read directly from Arweave without going through Tarn at all.

User-derived keys

All cryptographic material is derived from secrets the user holds: their email, password, and recovery phrase. Tarn cannot reset a password (the password *is* the key, not a check against a stored hash) and cannot recover an account on behalf of a user.

Per-app isolation

The same email and password used in two different apps produce two fully independent identities. A user's Bookish account and their Cellar account share no derivable keys, no lookup tags, and no observable linkage on Arweave.

Permanence over deletion

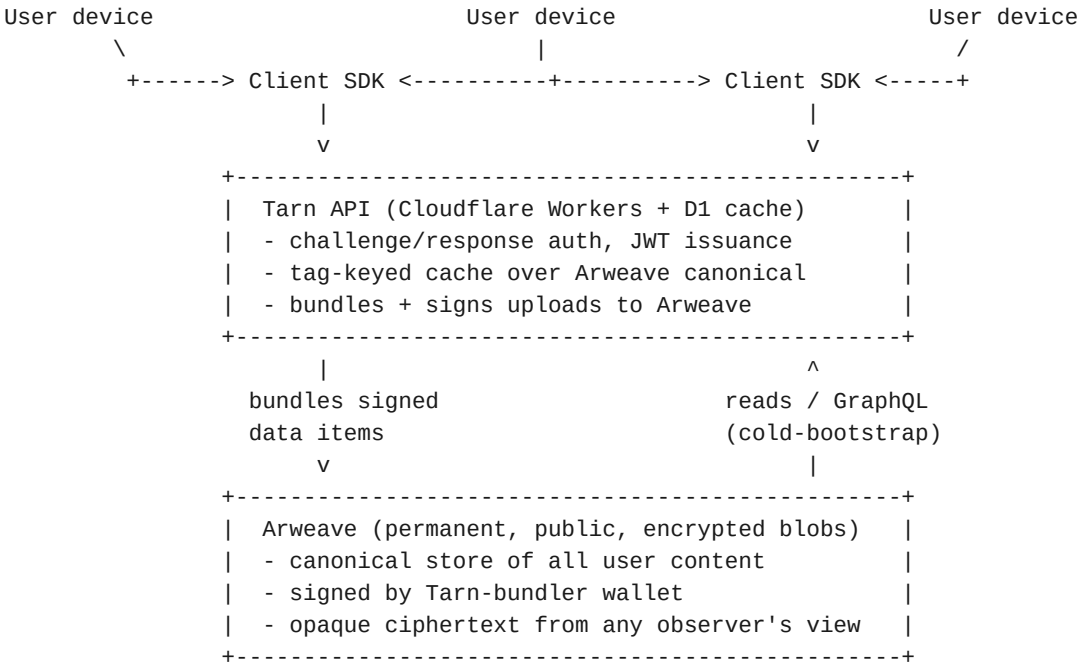
Data on Arweave is immutable. Tarn does not pretend otherwise. Users can stop publishing, rotate keys forward to make old data inaccessible to future credentials, or unfollow connections — but they cannot retroactively un-publish what was already encrypted to a recipient. The protocol and UX make this property explicit rather than hiding it.

Honest about residual leaks

Tarn does not promise more than it can deliver. Some metadata (connection-request timing, account existence via discoverability lookup) is observable to interested parties; this is documented explicitly in protocol notes rather than papered over.

3. System architecture

Tarn has four layers. The bottom two are what makes data permanent and tamper-evident; the top two are what makes it usable.



Layer 1 — Arweave (canonical store)

All user content lives on Arweave as encrypted blobs. Once written, blobs are permanent and tamper-evident. Tarn signs uploads via a single shared bundler wallet, which means Arweave-level observers see *that* Tarn customer activity is happening but cannot link writes to specific users without help from the protocol layer.

Layer 2 — Tarn API (cache + write proxy)

Cloudflare Workers serve as the read/write hot path. A D1 database caches Arweave blobs by tag for sub-100ms reads. The API does **not** see plaintext — it stores opaque ciphertext bytes — and is structurally rebuildable from Arweave (cold-bootstrap path included).

The API also handles authentication challenge-response, JWT issuance, rate limiting, and per-app rules enforcement. Apps register with Tarn and prove their identity via ECDSA challenge-response; user accounts do the same with their per-app signing key.

Layer 3 — Client SDK

All cryptography happens in the SDK. Key derivation, content encryption, share-log signing, recovery PDF rendering, BIP39 phrase generation, HPKE handshakes — none of this is network-mediated. The SDK is published as a JavaScript library and runs in browsers and Node.js.

Layer 4 — Application code

Applications (e.g., Bookish, the first app on Tarn) build their UX on top of the SDK. The SDK is intentionally product-neutral: it exposes primitives like "create encrypted entry", "share content with a connection", "read share log". Application code decides what to call those primitives in the user-facing UI.

4. Account model

An account in Tarn is identified by an (email, app_id) pair. The same email used across multiple apps produces multiple unrelated identities. Authentication uses a challenge-response signature; passwords are never transmitted to the server.

Key derivation chain

From the user's two secrets — email and password — Tarn derives every key the system needs:

```
email + password
      |
      v   Argon2id (m=64MiB, t=3, p=1, salt=SHA-256(email))
      |
  master_key  (32 bytes, never leaves device)
      |
      +-- HKDF(info="lookup")  --> credential_lookup_key  (server pseudonym)
      +-- HKDF(info="encrypt") --> credential_encryption_key (KEK)
      +-- HKDF(info="sign")    --> ECDSA P-256 signing keypair
      +-- HKDF(info="share")   --> X25519 sharing keypair

Every HKDF call also includes app_id, so all sub-keys are app-scoped.
```

The key derivation function is Argon2id, a memory-hard KDF that resists brute force from GPUs and ASICs. Legacy accounts on PBKDF2 continue to work via a fallback path; new accounts always use Argon2id. The KDF version is recorded in the user's credential blob so the client knows which to use on subsequent logins.

Per-app isolation

Every derivation embeds app_id in its HKDF info string. This means the same email + password used in app A and app B produce completely different lookup keys, encryption keys, signing keys, and sharing keys. From the protocol's view, the two accounts are unrelated. An Arweave observer scanning tags cannot link them; Tarn the service sees two separate accounts with no derivable connection.

Authentication

Login is a challenge-response cycle. The client requests a challenge (receives a fresh nonce), signs the nonce with its app-scoped ECDSA private key, and returns the signature. The server verifies against the stored public key and issues a short-lived JWT. The user's password is never transmitted; the server stores no password hash.

> *— Tarn cannot "reset" a password the way a traditional service can. There is no stored hash to compare against and no out-of-band recovery channel under Tarn's control. The password is half of the user's key material. The other half — the recovery phrase — exists for exactly this reason.*

5. Data encryption

User content is encrypted twice over: once with a per-content key (the CEK), which is itself wrapped under a generation-indexed data encryption key (the DEK). This two-layer pattern enables selective sharing and forward-secret rotation without re-encrypting existing data.

The DEK chain

Each user has a chain of DEKs, one per generation. New accounts start at gen 1 with a randomly generated 32-byte key. On every credential change, a fresh DEK is appended at gen N+1. Old generations stay in the chain so that data written under them is still readable; new writes use the latest generation.

The chain is wrapped under the user's credential_encryption_key (KEK) and stored in their credential blob on Arweave. Multi-factor wrapping: every chain entry is wrapped twice — once under the password-derived KEK, once under a recovery-phrase-derived KEK. Either factor independently unwraps the chain.

Per-content CEKs

Each content blob carries its own CEK, randomly generated, used once to encrypt that blob's plaintext. The CEK is wrapped under the current-generation DEK and prepended to the ciphertext along with a 5-byte format magic prefix:

```
data_blob = magic(5) || wrapped_CEK(40) || iv(12) || ciphertext+tag
where:
    magic       = 0x54 0x41 0x52 0x4e 0x02   // ASCII "TARN" + version
    wrapped_CEK = AES-KW(DEK_at_current_gen, CEK)
    iv          = random_bytes(12)
    ciphertext  = AES-256-GCM(CEK, iv, plaintext)
```

The owner reads by unwrapping the CEK with their DEK and decrypting the ciphertext. A recipient who has been given the CEK directly (via the sharing layer — see §7) skips the wrapped_CEK portion and decrypts the same ciphertext blob with the CEK they were given. The owner never exposes their DEK; only specific CEKs are shared.

Forward-secret rotation

When a user changes their password, the new credentials cannot decrypt data written before the change unless the user explicitly carries the old DEKs forward (which Tarn does, by design, so existing data stays readable). But future writes use a fresh DEK at the new generation, and an attacker with only the old credentials cannot derive that new DEK. The blast radius of a credential leak is bounded to data written before the rotation.

6. Account recovery

Tarn issues every user a 24-word recovery phrase at signup. This phrase is generated client-side from cryptographic randomness, never seen by Tarn, and serves as a parallel access path to the user's data: a different KEK that can independently unwrap the same DEK chain that the password-derived KEK protects.

How the phrase is delivered

At signup the SDK generates the phrase, renders a printable PDF containing the phrase + instructions, and (by default, opt-in) emails a copy to the user. Tarn's API briefly handles the PDF in memory while forwarding it via an external email relay; nothing is persisted. The user is required to acknowledge that they have saved the phrase before the registration completes.

How recovery works

If the user later loses their password, they enter the 24-word phrase into the SDK. The phrase derives a recovery KEK (Argon2id over the phrase + a per-account salt stored in the credential blob), and from there a recovery-specific lookup key and signing key. The user authenticates with the recovery signing key, unwraps the DEK chain via the recovery factor, sets new credentials, and re-wraps the chain under the new password KEK. All existing data is preserved; new credentials work going forward.

> *— Email + password change is a routine convenience. Account recovery via phrase is the response to suspected compromise. Apps building on Tarn should communicate this distinction to users.*

7. Sharing and connections

Users can selectively share encrypted content with other users, end-to-end. The sharing layer is built on the same primitives as the private content layer — Arweave for storage, per-content CEKs for encryption — extended with a few additional keys.

Connections, not friends or follows

The Tarn protocol calls a relationship between two users a *connection*. The term is intentionally neutral: applications can present connections as friends, followers, contacts, or however suits their UX. The protocol-level relationship is mutual — both sides explicitly accept — and apps deliver asymmetric "follow" UX on top by combining the mutual primitive with a per-side mute filter.

Connection handshake

Establishing a connection is a two-step exchange:

- Alice sends a connection request to Bob: an HPKE-sealed payload (RFC 9180, DHKEM-X25519 + HKDF-SHA-256 + AES-256-GCM) addressed to Bob's published sharing public key.
- Bob accepts: an HPKE-sealed acceptance addressed to Alice's sharing public key, referencing the original request's nonce.

After both messages are exchanged, both users add each other to their *connections record* — an encrypted Tarn data blob that lists their accepted connections. From this point on, both can publish share-log entries to each other.

Per-pair share log

Each connection has two append-only share logs, one in each direction. Each log entry is encrypted under a per-pair AES-GCM key derived from the X25519 ECDH shared secret between the two parties' sharing keypairs. Entries are signed with the sender's identity key, and tagged on Arweave with stealth-addressed identifiers (HMAC under a per-pair seed) — meaning that an Arweave observer cannot link tags to specific user pairs without that seed.

The five operation types are add (first-share a content item), update (publish a new version), rotate (change the per-content CEK), remove (unshare), and snapshot (full-state checkpoint for fast bootstrap). A sixth operation, rotate_identity, handles credential changes — see §8.

Why per-pair instead of broadcast

Per-pair encryption means each share is encrypted individually for each recipient. This scales gracefully to hundreds of connections and preserves strong metadata privacy — the protocol does not reveal who shares what with whom, even to network observers. A separate broadcast primitive for influencer-scale public sharing is anticipated as future work; the current design optimizes for private-circle use cases where metadata privacy matters.

Mute filter for asymmetric UX

Apps that want to present connections as Strava-style asymmetric follow ("I follow you, but you don't have to follow me back") layer a per-side mute filter on top of the mutual handshake. Muting a connection hides their content from the muting party's feed without affecting the underlying cryptographic relationship — the filter is purely a UI convention, persisted as an encrypted record so it syncs across the user's devices.

8. Identity rotation

When a user changes their email or password, every key derived from their master key changes too — including their sharing keypair. This would normally break all existing connections: Alice's old sharing private key is gone, so the per-pair shared secrets her contacts had with her no longer work.

To handle this transparently, Tarn uses a *rotation announcement* protocol. As part of the credential change, the SDK iterates over the user's connections and publishes a special rotate_identity operation to each connection's old share log, signed with the old signing key (which the SDK still has in memory) and containing the new sharing and signing public keys.

The recipient picks up the announcement on their next read of the share log, verifies the signature against the cached old signing key, updates their connections record with the new public keys, and transparently switches to the new per-pair derivation for all future reads and writes. From the user's perspective, the credential change is a no-op: their connections are preserved, their data is still readable, and their contacts auto-reconcile.

> *— The rotation flow assumes the old keys are not compromised at the moment of rotation. If they are, an attacker with the old keys could race the legitimate user and publish a fake rotation announcement to hijack the relationship. This limitation is documented; the right response to suspected compromise is to use full account recovery via the phrase, which rebuilds the identity independently of the old credentials. Future hardening (a separate rotation key derived from the recovery phrase) is on the roadmap.*

9. Session persistence

By default a logged-in Tarn client holds its derived keys — signing keypair, unwrapped DEK chain, sharing keypair, JWT — in memory only. When the page closes, the keys are gone, and the next session requires re-deriving from email + password (paying the full Argon2id cost). For consumer apps this is a UX floor that's hard to ship below.

Tarn provides an *opt-in* session-persistence primitive: apps that want it can serialize a logged-in client to an opaque blob, store the blob in localStorage, and resume from it on a later page load without prompting the user for their password. The default behaviour (no persistence) is unchanged; apps with stricter postures stay opted out.

At-rest encryption

The serialized session is encrypted under an AES-256-GCM wrapping key stored in IndexedDB with extractable: false. The non-extractable flag is the load-bearing piece of the threat model: even an attacker with code execution on the origin (XSS) cannot exfiltrate the raw key bytes for offline replay on another device — they can only invoke the key in-page, and only while they hold execution. The persisted blob is unreadable off-origin.

Threat model and trade-offs

Persisting derived keys to client-side storage is a meaningful change to Tarn's threat surface, and Tarn is honest about it. Without persistence, a same-origin XSS can act as the user only while the page is open. With persistence, the same XSS gains pseudo-persistent access: it can decrypt the persisted blob in-page and act as the user up to the blob's expiry.

Three constraints bound this risk:

- **Hard 7-day max age**, baked into the blob at creation. The SDK does not refresh expiry on use; a quiet exfil-and-replay attack is bounded to one week regardless of activity.
- **Origin binding via the non-extractable wrapping key.** Stealing the persisted blob is useless without simultaneous code execution on the origin.
- **Automatic invalidation on key rotation.** Credential change, account recovery, and account deletion each rotate the wrapping key as a side effect, rendering all previously-emitted blobs on the origin unreadable.

Apps with stricter threat models (financial, medical, etc.) should simply not opt in. The default constructor + login() path persists nothing.

What gets persisted

Just enough to reconstitute the in-memory client without re-deriving from password: the signing keypair (PKCS#8 + SPKI), the unwrapped DEK chain (raw bytes per generation), the sharing keypair, the data and credential lookup keys, the recovery-factor metadata used by changeCredentials, and the current JWT. The credential_encryption_key is intentionally omitted — once the DEK chain is unwrapped at login, the KEK is dead state. Share-log caches are not persisted; they re-hydrate from Arweave on first use.

Server-side session management

Section 7 (above) covers persistence — keeping a logged-in client alive across page reloads on a single device. Section 7.5 covers the complementary capability across devices: *multi-device session management* — letting a user list active sessions and revoke any of them individually without performing a full credential change.

Shape:

- **Per-session identifier.** Every successful /auth/verify issues a JWT carrying a sid claim — a fresh UUID. The server records the session in a new D1 sessions table with created_at, last_seen_at, and an optional device label.
- **Revocation endpoints.** DELETE /api/v1/sessions/:sid kills one session; DELETE /api/v1/sessions kills all active sessions for the user. Both require a valid JWT from any session belonging to the same account.
- **Listing endpoint.** GET /api/v1/sessions returns the user's active sessions so an app can render a *Manage devices* page.
- **Stateful auth middleware.** Authenticated request handling consults the sessions table to confirm the JWT's sid is still active. The added D1 lookup is mitigated by a short in-Worker cache to avoid hot-pathing the database.

Together, persistence (Section 7) and session management (Section 7.5) close the consumer-app session story: users stay logged in across reloads on the devices they trust, and can kill individual sessions cleanly when they don't. changeCredentials remains available as a heavy-hammer alternative — it rotates the signing key and locks out every other device by force — but for routine

device management the granular revoke endpoints are the right tool.

10. Invite tokens

Section 7 (sharing) lets two users form an end-to-end-encrypted connection if and only if the sender knows the recipient's email and the recipient is already a Tarn user with discoverability enabled. For consumer apps the prevailing UX is the inverse: scan a QR, click a link in a messenger, register-then-redeem. Section 10 adds an **opaque, single-use, time-limited invite token** primitive that lets the inviter publish a redemption slot without knowing the recipient's identity, and lets the recipient redeem it (potentially after signing up) without ever transmitting the inviter's identifier through the channel that carried the link.

Why server-mediated

Three alternative shapes were considered and rejected:

- **Stateless QR / link.** Keys baked into the URL, no server state. Loses single-use; a leaked link in any messenger channel exposes the inviter to arbitrary stranger redemption forever.
- **Pure-Arweave invite blob.** Inviter publishes an Arweave entry; recipient reads it and sends a normal connection request. Single-use cannot be enforced at the storage layer; app-side single-use races between the legitimate recipient and any malicious party that scrapes the link.
- **Reuse the existing inbox-tag mechanism.** The inbox is share_pub-keyed and time-windowed; an anonymous inbox without a share_pub doesn't fit the model, and we'd be inventing single-use semantics on a primitive that doesn't want them.

All three lose the atomic single-use property that only the server can provide cheaply. Section 10 is server-mediated for that reason alone — every other piece of the flow is client-side cryptography over an opaque blob.

Cryptographic shape

The inviter generates two independent 32-byte secrets client-side: token_id (the opaque server-side index) and payload_key (the AES-256-GCM key that encrypts the invite payload). The payload contains the inviter's share_pub, signing_pub, display_name, app id, and timestamp. The token_id goes in the URL path; the payload_key goes in the URL fragment, which browsers do not include in HTTP requests.

The Tarn API stores opaque ciphertext keyed on token_id and never sees the inviter's keys, name, or any other payload field. The recipient retrieves the ciphertext via an unauthenticated GET /api/v1/invite/:token_id for preview, decrypts client-side with the URL-fragment key, and then redeems via an authenticated POST /api/v1/invite/redeem/:token_id that atomically marks the token used. Both endpoints rate-limit via the existing primitives — D1-atomic for create/redeem (per account, per hour), KV-based for preview (per IP).

No Tarn-hosted landing page

Apps own the URL surface. Each registered app records an invite_url_template (e.g. https://app.bookish.example/invite/{token_id}); the SDK substitutes {token_id} and appends the URL fragment. The app's web handler reads the path + hash, calls the SDK, and renders whatever UX it wants — modal, toast, native deep-link. Tarn does not host any landing page. Messenger preview unfurls and install fallbacks are the app's responsibility.

Composes with existing connection primitives

After redeeming, the recipient's SDK sends a normal connection-request HPKE-sealed back to the inviter using the keys retrieved from the decrypted payload. The request carries an extra via_invite_token field. The inviter's SDK, on its next listIncomingRequests poll, reads its issued-invites blob fresh and auto-accepts requests whose token matches a known issuance. Unmatched requests stay in the pending list for the user to act on. The end state is identical to a connection formed via email handshake — same connection record, same share-log mechanics, same mute and revoke primitives.

Section 10 also wires up Connection.label as a real primitive — previously documented as an optional field but never implemented. Invite redemption seeds the new connection's label from the inviter's display_name, so apps get an out-of-the-box *This is Maya* tag without app-layer storage. Apps can also set or update labels manually via setConnectionLabel.

Threat model

The dominant risk is a leaked link (screenshot, accidental Slack post). Mitigations: 7-day default expiry, hard 30-day max, single-use, and a sender-visible fingerprint of the redeemer's share_pub so the inviter can detect surprise redemption and revoke + reissue. A server compromise lets an attacker enumerate live tokens but not decrypt payloads — the decryption keys are URL fragments held only by recipients.

The server learns metadata: which account issued an invite, the time of issuance, the time and originating IP of redemption, and the redeemer's share_pub fingerprint. It does not learn the inviter's share_pub, signing public key, display name, or any payload contents — those are inside the encrypted blob keyed by a secret the server never sees. This is the same zero-knowledge story as the rest of the protocol.

11. Deferred features

Several capabilities are intentionally deferred from the v1 platform. They are documented here so that future evolution paths are visible.

Public broadcast / influencer scale

The current per-pair pattern strains around hundreds of connections per user. For applications that want public profiles where anyone can subscribe without approval, a separate broadcast primitive — single publish, fan-out reads — would be added. The cryptographic shape is different (publicly-readable signed content rather than per-recipient encryption); both can coexist in the same Tarn account.

Hardened rotation under compromised keys

A separate rotation signing key derived from the recovery phrase (and thus independent of the master key) would close the §8 limitation. Adds modest complexity; not v1.

Forward secrecy on sharing

Signal-style prekey bundles would give per-message forward secrecy for share-log entries, so that a future compromise of the recipient's key cannot decrypt past traffic. Achievable Arweave-native; complex; deferred until a use case demands it.

Decoy traffic for stronger metadata privacy

Padding share log writes with decoy entries would obscure inferences from observable wrapping patterns. Substantial cost for marginal improvement; deferred.

12. Glossary

Term	Meaning
DEK	Data Encryption Key. Per-user, randomly generated, wraps per-content CEKs. Generation-indexed.
CEK	Content Encryption Key. Per-blob, randomly generated, used once. Stable across versions of the same
KEK	Key Encryption Key. Derived from password (or recovery phrase). Wraps the DEK chain.
Argon2id	Memory-hard password-based KDF. Used for password → master_key.
HKDF	Hash-based Key Derivation Function (RFC 5869). Used to expand master_key into all sub-keys.
HPKE	Hybrid Public-Key Encryption (RFC 9180). Used for connection-handshake bootstrap.
X25519	Elliptic curve Diffie-Hellman over Curve25519. Used for sharing keypair and per-pair shared secrets.
Connection	A mutual cryptographic relationship between two users. Apps may present this as friend/follow/contact/
Share log	Per-pair append-only stream of share operations. Stealth-addressed. Encrypted under per-pair K_AB.
Stealth address	Tag derived via HMAC under a per-pair secret. Unlinkable to outside observers.
Recovery phrase	24-word BIP39 mnemonic generated client-side at signup. Independent access path to the DEK chain.

13. Where to learn more

Protocol specification. The canonical reference is docs/TARN_PROTOCOL.md in the Tarn repository — full key-derivation derivations, blob formats, API surface, and rotation flows.

SDK reference. The client SDK's README (client/README.md) covers the JavaScript API surface with examples: register, login, recovery, content CRUD, connection lifecycle, sharing primitives, mute filter.

Design history. The two design notes (2026-04-28-tarn-account-and-data-model.md and 2026-04-28-tarn-sharing-design.md) captured the design rationale during the build. They are useful reading for understanding *why* certain decisions were made — particularly the trade-offs around revocation, metadata privacy, and recovery.

Implementation history. Issues 9–22 on the Tarn repository, filed and closed during the platform build, document the unit of work for each protocol section. Each closing comment serves as a self-contained record of what shipped under that section.